Mini Project 12 - Melatih Tiny CNN dari Log-Mel

Tujuan

adalah menghasilkan satu CNN kecil yang dibekukan berdasarkan validation set. Mini Project ini tidak membuka test set dan belum melakukan quantization/deployment final. Outputnya menjadi kandidat deep learning yang akan dibandingkan dengan baseline klasik pada Mini Project 13.

A. Prasyarat

- Mini Project 11 selesai dan project_status.json bernilai ready_for_mp12.
- Folder features/logmel/mp09 berisi log-Mel seluruh 24 sampel dari MP9.
- splits/mp09_split.json tetap memakai block 1-4 training, block 5 validation, block 6 test.
- config/classifier_config.json tersedia sehingga urutan kelas konsisten dengan MP11.
- Python environment telah memiliki numpy, matplotlib, scikit-learn, dan torch.

B. Langkah 1 - Validasi Input Serial

Buka terminal pada root running project. Jangan membuat folder proyek baru. Jalankan pemeriksaan berikut:

```
python - <<'PY'
import json, glob
s=json.load(open('project_status.json'))
print('status =', s['status'])
print('jumlah log-Mel =', len(glob.glob('features/logmel/mp09/*.npz')))
sp=json.load(open('splits/mp09_split.json'))
print('train/val/test =', len(sp['train_ids']), len(sp['validation_ids']), len(sp['test_ids']))
PY
```

Hasil minimum yang diterima adalah status ready_for_mp12, jumlah log-Mel 24, dan split 16/4/4.

Pemeriksaan	Harus bernilai	Jika gagal
project_status	ready_for_mp12	Kembali ke MP11
log-Mel files	24	Periksa MP9
train/val/test	16/4/4	Jangan membuat split baru
test prediction MP11	false	Jangan lanjut bila test pernah dipakai untuk tuning

Tabel 12.12. Gate sebelum training deep learning.

C. Langkah 2 - Simpan Kode Python

Buat file mini_project_12.py pada root project, kemudian salin kode berikut. Kode memotong atau melakukan padding log-Mel ke 96 frame, menghitung normalisasi hanya dari training set, memberi augmentasi hanya pada training, memilih checkpoint dari validation, menghitung resource proxy, dan memperbarui status proyek.

```
from pathlib import Path
import json, random, math, copy
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from sklearn.metrics import f1_score, accuracy_score, confusion_matrix

ROOT = Path('.')
SEED = 2026
EPOCHS = 60
BATCH_SIZE = 8
LR = 1e-3
PATIENCE = 12
TARGET_FRAMES = 96

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.set_num_threads(1)
```

```
def save_json(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, indent=2), encoding='utf-8')


def macro_f1(y_true, y_pred):
    return float(f1_score(y_true, y_pred, average='macro', zero_division=0))


def load_logmel(sample_id):
    p = ROOT / 'features/logmel/mp09' / f'{sample_id}.npz'
    if not p.exists():
        raise FileNotFoundError(p)
    d = np.load(p, allow_pickle=True)
    x = d['log_mel'].astype(np.float32)
    # Samakan panjang waktu tanpa mengubah jumlah Mel band.
    if x.shape[0] >= TARGET_FRAMES:
        x = x[:TARGET_FRAMES]
    else:
        pad = TARGET_FRAMES - x.shape[0]
        fill = float(np.min(x)) if x.size else 0.0
        x = np.pad(x, ((0,pad),(0,0)), mode='constant', constant_values=fill)
    label = str(d['label'])
    return x, label


status_path = ROOT / 'project_status.json'
if not status_path.exists():
    raise FileNotFoundError('project_status.json tidak ditemukan.')
status = json.loads(status_path.read_text(encoding='utf-8'))
if status.get('status') != 'ready_for_mp12':
    raise RuntimeError(f"Status harus ready_for_mp12, ditemukan {status.get('status')!r}")

split_path = ROOT / 'splits/mp09_split.json'
split = json.loads(split_path.read_text(encoding='utf-8'))
train_ids = split['train_ids']
val_ids = split['validation_ids']
test_ids = split['test_ids']  # hanya dicatat; tidak dimuat/prediksi pada MP12

# Kelas dibekukan dari MP11 agar encoding konsisten.
clf_cfg = json.loads((ROOT/'config/classifier_config.json').read_text(encoding='utf-8'))
classes = list(clf_cfg['classes'])
class_to_idx = {c:i for i,c in enumerate(classes)}

# 1) Muat HANYA training dan validation log-Mel.
train_x, train_y = [], []
for sid in train_ids:
    x, y = load_logmel(sid)
    train_x.append(x); train_y.append(class_to_idx[y])
val_x, val_y = [], []
for sid in val_ids:
    x, y = load_logmel(sid)
    val_x.append(x); val_y.append(class_to_idx[y])
train_x = np.stack(train_x)
val_x = np.stack(val_x)
train_y = np.asarray(train_y, dtype=np.int64)
val_y = np.asarray(val_y, dtype=np.int64)

# 2) Normalisasi global dihitung dari TRAINING ONLY.
mu = float(train_x.mean())
sigma = float(train_x.std() + 1e-6)
train_x = (train_x - mu) / sigma
val_x = (val_x - mu) / sigma

class LogMelDataset(Dataset):
    def __init__(self, X, y, augment=False):
        self.X = X
        self.y = y
        self.augment = augment
    def __len__(self):
        return len(self.y)
    def __getitem__(self, i):
        x = self.X[i].copy()
        if self.augment:
            # Augmentasi ringan hanya pada training: gain/noise dan masking kecil.
            x *= np.random.uniform(0.95, 1.05)
            x += np.random.normal(0, 0.015, size=x.shape).astype(np.float32)
            if np.random.rand() < 0.5:
                t0 = np.random.randint(0, max(1, x.shape[0]-6))
                x[t0:t0+4, :] = 0
            if np.random.rand() < 0.5:
                f0 = np.random.randint(0, max(1, x.shape[1]-4))
                x[:, f0:f0+3] = 0
        return torch.from_numpy(x[None, ...]).float(), int(self.y[i])
```

```
train_loader = DataLoader(LogMelDataset(train_x, train_y, True), batch_size=BATCH_SIZE,
                          shuffle=True, generator=torch.Generator().manual_seed(SEED))
val_loader = DataLoader(LogMelDataset(val_x, val_y, False), batch_size=BATCH_SIZE, shuffle=False)

class TinyKWSNet(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(8), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(16), nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1)),
        )
        self.classifier = nn.Linear(16, n_classes)
    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)

model = TinyKWSNet(len(classes))
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)

best_state = None
best_epoch = 0
best_f1 = -1.0
best_loss = float('inf')
no_improve = 0
history = {'train_loss':[], 'val_loss':[], 'val_accuracy':[], 'val_macro_f1':[]}

for epoch in range(1, EPOCHS+1):
    model.train(); total_loss = 0.0; n = 0
    for xb, yb in train_loader:
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward(); optimizer.step()
        total_loss += float(loss.item()) * len(yb); n += len(yb)
    train_loss = total_loss / max(n,1)

    model.eval(); vloss = 0.0; vn = 0; preds=[]; refs=[]
    with torch.no_grad():
        for xb, yb in val_loader:
            logits = model(xb)
            loss = criterion(logits, yb)
            vloss += float(loss.item()) * len(yb); vn += len(yb)
            preds.extend(logits.argmax(1).cpu().numpy().tolist())
            refs.extend(yb.cpu().numpy().tolist())
    val_loss = vloss / max(vn,1)
    val_acc = float(accuracy_score(refs, preds))
    val_f1 = macro_f1(refs, preds)
    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['val_accuracy'].append(val_acc)
    history['val_macro_f1'].append(val_f1)

    # Checkpoint berdasarkan macro-F1, loss sebagai tie-break.
    improved = (val_f1 > best_f1 + 1e-9) or (abs(val_f1-best_f1) < 1e-9 and val_loss < best_loss)
    if improved:
        best_f1 = val_f1; best_loss = val_loss; best_epoch = epoch
        best_state = copy.deepcopy(model.state_dict()); no_improve = 0
    else:
        no_improve += 1
    if no_improve >= PATIENCE and epoch >= 20:
        break

model.load_state_dict(best_state)
model.eval()
with torch.no_grad():
    Xv = torch.from_numpy(val_x[:,None,:,:]).float()
    val_logits = model(Xv)
    val_pred = val_logits.argmax(1).cpu().numpy()
val_cm = confusion_matrix(val_y, val_pred, labels=np.arange(len(classes)))

# 3) Resource proxy. Test tetap tidak dibaca.
param_count = sum(p.numel() for p in model.parameters())
float32_bytes = int(param_count * 4)
int8_weight_bytes_est = int(param_count)  # estimasi, bukan hasil quantization/deployment.
# MAC proxy untuk input T x F: conv1 + pool + conv2 + FC.
T, F = train_x.shape[1], train_x.shape[2]
conv1_macs = T * F * 8 * 1 * 3 * 3
T2, F2 = T//2, F//2
conv2_macs = T2 * F2 * 16 * 8 * 3 * 3
fc_macs = 16 * len(classes)
macs = int(conv1_macs + conv2_macs + fc_macs)
```

```
# 4) Simpan model dan konfigurasi. Tidak ada test prediction.
model_dir = ROOT/'models'; model_dir.mkdir(parents=True, exist_ok=True)
fig_dir = ROOT/'results/figures'; fig_dir.mkdir(parents=True, exist_ok=True)
metric_dir = ROOT/'results/metrics'; metric_dir.mkdir(parents=True, exist_ok=True)
config_dir = ROOT/'config'; config_dir.mkdir(parents=True, exist_ok=True)

model_path = model_dir/'mp12_tiny_cnn_state.pt'
torch.save({'state_dict': model.state_dict(), 'classes': classes,
        'normalization': {'mean':mu,'std':sigma}}, model_path)

# Simpan parameter ke NPZ agar struktur bobot mudah diaudit pada bab deployment.
npz_payload = {}
for name, tensor in model.state_dict().items():
   npz_payload[name.replace('.','__')] = tensor.detach().cpu().numpy()
npz_payload['classes'] = np.asarray(classes, dtype=object)
npz_payload['normalization_mean'] = np.asarray([mu], dtype=np.float32)
npz_payload['normalization_std'] = np.asarray([sigma], dtype=np.float32)
np.savez_compressed(model_dir/'mp12_tiny_cnn_parameters.npz', **npz_payload)

config = {
   'input_representation': 'log_mel',
   'source_logmel_dir': 'features/logmel/mp09',
   'split_source': 'splits/mp09_split.json',
   'architecture': 'TinyKWSNet_2conv_gap',
   'input_shape_per_sample': [int(T), int(F)],
   'classes': classes,
   'normalization_fit_scope': 'training_only',
   'augmentation_scope': 'training_only',
   'checkpoint_selection_scope': 'validation_only',
   'test_files_loaded': False,
   'test_predictions_generated': False,
   'seed': SEED,
   'batch_size': BATCH_SIZE,
   'learning_rate': LR,
   'max_epochs': EPOCHS,
   'early_stopping_patience': PATIENCE,
   'best_epoch': int(best_epoch),
   'model_file': str(model_path),
   'parameter_file': 'models/mp12_tiny_cnn_parameters.npz',
   'quantization_performed': False,
   'quantization_deferred_to_mp15': True
}
save_json(config_dir/'deep_learning_config.json', config)

report = {
   'n_train': int(len(train_y)),
   'n_validation': int(len(val_y)),
   'n_test_reserved_untouched': int(len(test_ids)),
   'test_ids_loaded_as_features': False,
   'test_predictions_generated': False,
   'input_shape': [int(T), int(F)],
   'parameter_count': int(param_count),
   'float32_parameter_bytes': float32_bytes,
   'estimated_int8_parameter_bytes': int8_weight_bytes_est,
   'estimated_macs_per_inference': macs,
   'best_epoch': int(best_epoch),
   'validation_macro_f1': float(best_f1),
   'validation_accuracy': float(accuracy_score(val_y, val_pred)),
   'validation_confusion_matrix': val_cm.tolist(),
   'history': history,
   'note': 'Angka resource adalah proxy arsitektur. Quantization dan benchmark perangkat dilakukan pada MP15.'
}
save_json(metric_dir/'mp12_deep_learning_metrics.json', report)

# Learning curve
fig, ax = plt.subplots(figsize=(7.2,4.5))
e = np.arange(1, len(history['train_loss'])+1)
ax.plot(e, history['train_loss'], label='train loss')
ax.plot(e, history['val_loss'], label='validation loss')
ax.axvline(best_epoch, linestyle='--', label='best checkpoint')
ax.set_xlabel('Epoch'); ax.set_ylabel('Cross-entropy loss'); ax.set_title('MP12 learning curve')
ax.grid(alpha=.2); ax.legend(); fig.tight_layout(); fig.savefig(fig_dir/'mp12_learning_curve.png', dpi=170); plt.close(fig)

fig, ax = plt.subplots(figsize=(5.2,4.6))
im=ax.imshow(val_cm, cmap='Blues')
ax.set_xticks(range(len(classes)), classes, rotation=30, ha='right')
ax.set_yticks(range(len(classes)), classes)
ax.set_xlabel('Prediksi'); ax.set_ylabel('Aktual'); ax.set_title('MP12 validation confusion matrix')
for i in range(len(classes)):
   for j in range(len(classes)):
      ax.text(j,i,str(val_cm[i,j]),ha='center',va='center')
fig.colorbar(im, ax=ax, fraction=.046, pad=.04); fig.tight_layout(); fig.savefig(fig_dir/'mp12_validation_confusion_matrix.png', dpi=170); plt.close(fig)

# Resource card/bar
fig, ax = plt.subplots(figsize=(7.2,4.3))
labels=['Parameters','Float32 bytes','Est. INT8 bytes','MACs / inference']
vals=[param_count,float32_bytes,int8_weight_bytes_est,macs]
```

```
ax.bar(labels, vals)
ax.set_yscale('log'); ax.set_ylabel('Count / bytes (log scale)'); ax.set_title('MP12 resource proxy')
for i,v in enumerate(vals): ax.text(i,v*1.08,f'{v:,}',ha='center',fontsize=9)
ax.tick_params(axis='x',rotation=18); fig.tight_layout(); fig.savefig(fig_dir/'mp12_resource_proxy.png', dpi=170); plt.close(fig)

status['status'] = 'ready_for_mp13'
status['last_completed_mini_project'] = 12
status['mp12'] = {
  'deep_learning_config': 'config/deep_learning_config.json',
  'model_file': str(model_path),
  'parameter_file': 'models/mp12_tiny_cnn_parameters.npz',
  'metrics': 'results/metrics/mp12_deep_learning_metrics.json',
  'test_used': False,
  'quantization_deferred_to_mp15': True
}
save_json(status_path, status)

print('MP12 selesai')
print('best_epoch =', best_epoch)
print('validation_macro_f1 =', round(best_f1,4))
print('parameters =', param_count)
print('estimated_macs =', macs)
print('test predictions generated = False')
print('status = ready_for_mp13')
```

D. Langkah 3 - Jalankan Training

```
python mini_project_12.py
```

Training pada running project sengaja kecil. Jika perangkat komputer lambat, jangan menaikkan jumlah epoch atau ukuran kanal. Tujuan mini project adalah memahami workflow dan menghasilkan artefak serial, bukan mencari konfigurasi terbesar.

Output terminal yang normal antara lain menampilkan best_epoch, validation_macro_f1, parameter count, estimated MACs, test predictions generated = False, dan status = ready_for_mp13.

E. Langkah 4 - Periksa Learning Curve dan Validation

Buka dua gambar yang dihasilkan: results/figures/mp12_learning_curve.png dan results/figures/mp12_validation_confusion_matrix.png. Periksa apakah loss training turun, validation tidak mengalami kegagalan numerik, dan confusion matrix memiliki empat kelas. Dengan validation hanya satu sampel per kelas, jangan menarik kesimpulan statistik yang berlebihan.

F. Langkah 5 - Periksa Resource Proxy

Buka results/figures/mp12_resource_proxy.png dan results/metrics/mp12_deep_learning_metrics.json. Catat parameter_count, float32_parameter_bytes, estimated_int8_parameter_bytes, dan estimated_macs_per_inference. Nilai tersebut menjadi hipotesis awal resource, bukan benchmark perangkat.

Artefak	Isi	Dipakai lagi pada
config/deep_learning_config.json	Arsitektur, normalisasi, training policy	MP13 dan MP15
models/mp12_tiny_cnn_state.pt	Checkpoint float CNN	MP13
models/mp12_tiny_cnn_parameters.npz	Bobot numerik yang mudah diaudit	MP15
results/metrics/mp12_deep_learning_metrics.json	Validation dan resource proxy	MP13
mp12_learning_curve.png	Diagnostik training	Laporan
mp12_validation_confusion_matrix.png	Diagnostik validation	Laporan
project_status.json	ready_for_mp13	Gate MP13

Tabel 12.13. Artefak wajib yang dibawa ke Mini Project 13.

G. Langkah 6 - Validasi Handoff ke Mini Project 13

```
python - <<'PY'
import json, pathlib
s=json.load(open('project_status.json'))
assert s['status']=='ready_for_mp13'
assert s['mp12']['test_used'] is False
for p in ['config/deep_learning_config.json',
      'models/mp12_tiny_cnn_state.pt',
      'models/mp12_tiny_cnn_parameters.npz',
      'results/metrics/mp12_deep_learning_metrics.json']:
```

```
  assert pathlib.Path(p).exists(), p
print('MP12 VALID: siap Mini Project 13')
PY
```

Handoff Serial MP12 -> MP13

Mini Project 13 akan membuka test set satu kali untuk mengevaluasi baseline machine learning MP11 dan CNN MP12 yang sudah dibekukan. MP13 tidak boleh mengubah feature selection, hyperparameter, arsitektur CNN, normalisasi, atau checkpoint berdasarkan hasil test.

Referensi

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.

2. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521, 436-444.

3. Sainath, T. N., & Parada, C. (2015). Convolutional neural networks for small-footprint keyword spotting. In Interspeech.

4. Zhang, Y., Suda, N., Lai, L., & Chandra, V. (2017). Hello Edge: Keyword Spotting on Microcontrollers. arXiv:1711.07128.

5. Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861.

6. Chollet, F. (2017). Xception: Deep Learning with Depthwise Separable Convolutions. Proceedings of CVPR.

7. Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. ICLR.

8. Warden, P., & Situnayake, D. (2019). TinyML. O'Reilly Media.

9. Sze, V., Chen, Y.-H., Yang, T.-J., & Emer, J. S. (2020). Efficient Processing of Deep Neural Networks. Morgan & Claypool.

10. Rabiner, L. R., & Schafer, R. W. (2010). Theory and Applications of Digital Speech Processing. Pearson.